

Digital Image Processing

ASSIGNMENT -7

Perumalla Dharan

AP21110010201

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"E:\SRM\Digital Image\Lab 7 - 28th
October\Cameraman.png", 0)
M, N = img.shape
```

Q1) Develop programs to implement Gaussian, uniform and impulse noise.

```
gauss_noise=np.zeros((M,N),dtype=np.uint8)
cv2.randn(gauss_noise,128,20)
gauss_noise=(gauss_noise*0.5).astype(np.uint8)

# uni_noise=np.zeros((640,480),dtype=np.uint8)
uni_noise=np.zeros((M,N),dtype=np.uint8)
cv2.randu(uni_noise,0,255)
uni_noise=(uni_noise*0.5).astype(np.uint8)

# imp_noise=np.zeros((640,480),dtype=np.uint8)
imp_noise=np.zeros((M,N),dtype=np.uint8)
cv2.randu(imp_noise,0,255)
imp_noise=cv2.threshold(imp_noise,200,255,cv2.THRESH_BINARY)[1]
```

Q2) Add this to an image.

```
img1 = img + gauss_noise  
img2 = img + uni_noise  
img3 = img + imp_noise  
  
plt.imshow(img1, cmap='gray')  
plt.title('Gaussian Noise')  
plt.axis("off")
```

Gaussian Noise



```
plt.imshow(img2, cmap='gray')  
plt.title('Uniform Noise')  
plt.axis("off")
```

Uniform Noise



```
plt.imshow(img3, cmap='gray')  
plt.title('Impulse Noise')  
plt.axis("off")
```

Impulse Noise



Q3) Apply denoising using mean, median and gaussian filters in spatial domain. Compare using MSE.

```
mean_filtered = cv2.blur(img1, (5, 5))           # Mean filter
median_filtered = cv2.medianBlur(img1, 5)         # Median filter
gaussian_filtered = cv2.GaussianBlur(img1, (5, 5), 0) # Gaussian filter

# Function to calculate Mean Squared Error
def mse(original, denoised):
    return np.mean((original - denoised) ** 2)

# Calculate MSE for each filtered image
mse_mean = mse(img, mean_filtered)
mse_median = mse(img, median_filtered)
mse_gaussian = mse(img, gaussian_filtered)
plt.subplot(3, 2, 1)
plt.imshow(img, cmap='gray')
```



```
plt.title('Original Image')
plt.axis("off")

plt.subplot(3, 2, 2)
plt.imshow(img1, cmap='gray')
plt.title('Image with Gaussian Noise')
plt.axis("off")

plt.subplot(3, 2, 3)
plt.imshow(mean_filtered, cmap='gray')
plt.title(f'Mean Filtered (MSE: {mse_mean:.2f})')
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(median_filtered, cmap='gray')
plt.title(f'Median Filtered (MSE: {mse_median:.2f})')
plt.axis("off")

plt.subplot(3,2,5)
plt.imshow(gaussian_filtered, cmap='gray')
plt.title(f'Gaussian Filtered (MSE: {mse_gaussian:.2f})')
plt.axis("off")
```

Original Image



Image with Gaussian Noise



Mean Filtered (MSE: 92.61)



Median Filtered (MSE: 88.74)



Gaussian Filtered (MSE: 90.25)



```
mean_filtered = cv2.blur(img2, (5, 5))           # Mean filter
median_filtered = cv2.medianBlur(img2, 5)         # Median filter
gaussian_filtered = cv2.GaussianBlur(img2, (5, 5), 0) # Gaussian filter

# Function to calculate Mean Squared Error
def mse(original, denoised):
    return np.mean((original - denoised) ** 2)

# Calculate MSE for each filtered image
mse_mean = mse(img, mean_filtered)
mse_median = mse(img, median_filtered)
mse_gaussian = mse(img, gaussian_filtered)

plt.subplot(3, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis("off")
```

```
plt.subplot(3, 2, 2)
plt.imshow(img2, cmap='gray')
plt.title('Image with Uniform Noise')
plt.axis("off")

plt.subplot(3, 2, 3)
plt.imshow(mean_filtered, cmap='gray')
plt.title(f'Mean Filtered (MSE: {mse_mean:.2f})')
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(median_filtered, cmap='gray')
plt.title(f'Median Filtered (MSE: {mse_median:.2f})')
plt.axis("off")

plt.subplot(3, 2, 5)
plt.imshow(gaussian_filtered, cmap='gray')
plt.title(f'Gaussian Filtered (MSE: {mse_gaussian:.2f})')
plt.axis("off")
```

Original Image



Image with Uniform Noise



Mean Filtered (MSE: 104.05)



Median Filtered (MSE: 109.05)



Gaussian Filtered (MSE: 106.04)



```
mean_filtered = cv2.blur(img3, (5, 5))           # Mean filter
median_filtered = cv2.medianBlur(img3, 5)         # Median filter
gaussian_filtered = cv2.GaussianBlur(img3, (5, 5), 0) # Gaussian filter

# Function to calculate Mean Squared Error
def mse(original, denoised):
    return np.mean((original - denoised) ** 2)

# Calculate MSE for each filtered image
mse_mean = mse(img, mean_filtered)
mse_median = mse(img, median_filtered)
mse_gaussian = mse(img, gaussian_filtered)

plt.subplot(3, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis("off")
```

```
plt.subplot(3, 2, 2)
plt.imshow(img3, cmap='gray')
plt.title('Image with Impulse Noise')
plt.axis("off")

plt.subplot(3, 2, 3)
plt.imshow(mean_filtered, cmap='gray')
plt.title(f'Mean Filtered (MSE: {mse_mean:.2f})')
plt.axis("off")

plt.subplot(3, 2, 4)
plt.imshow(median_filtered, cmap='gray')
plt.title(f'Median Filtered (MSE: {mse_median:.2f})')
plt.axis("off")

plt.subplot(3, 2, 5)
plt.imshow(gaussian_filtered, cmap='gray')
plt.title(f'Gaussian Filtered (MSE: {mse_gaussian:.2f})')
plt.axis("off")
```

Original Image



Image with Impulse Noise



Mean Filtered (MSE: 31.45)



Median Filtered (MSE: 24.41)



Gaussian Filtered (MSE: 24.97)



Q4) Convert the noisy image in frequency domain. Remove the three types of noise using ideal, Butterworth and gaussian filters. Compare using MSE.

```
from numpy.fft import fft2, fftshift, ifft2, ifftshift
# Fourier Transform
f_transform = fft2(img)
f_transform_shifted = fftshift(f_transform)
magnitude_spectrum = np.log(np.abs(f_transform_shifted) + 1)

# Display the original image and its Fourier Transform
plt.figure(figsize=(12,6))
plt.subplot(1,2,1), plt.imshow(img, cmap='gray'),
plt.title("Original Image")
plt.subplot(1,2,2), plt.imshow(magnitude_spectrum, cmap='gray'),
plt.title("Magnitude Spectrum")
plt.show()

# Ideal High-Pass Filter
```

```

def ideal_high_pass_filter(shape, D0):
    M, N = shape
    U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
    D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
    H = np.where(D > D0, 1, 0).astype(np.float32)
    return H

def butterworth_high_pass_filter(shape, D0, n):
    M, N = shape
    H = np.zeros((M, N), dtype=np.float32)
    epsilon = 1e-5 # Small value to avoid division by zero
    for u in range(M):
        for v in range(N):
            D = np.sqrt((u - M / 2) ** 2 + (v - N / 2) ** 2)
            if D == 0: # Handle the center (DC component)
                H[u, v] = 0
            else:
                H[u, v] = 1 / (1 + (D0 / (D + epsilon)) ** (2 *
n)) # Avoid zero in the denominator
    return H

# Gaussian High-Pass Filter
def gaussian_high_pass_filter(shape, D0):
    M, N = shape
    U, V = np.meshgrid(np.arange(M), np.arange(N),
indexing='ij')
    D = np.sqrt((U - M/2)**2 + (V - N/2)**2)
    H = 1 - np.exp(-(D ** 2) / (2 * (D0 **
2))) .astype(np.float32)
    return H

# Apply a filter and perform inverse Fourier Transform
def apply_filter_and_inverse(f_transform_shifted, H):

```



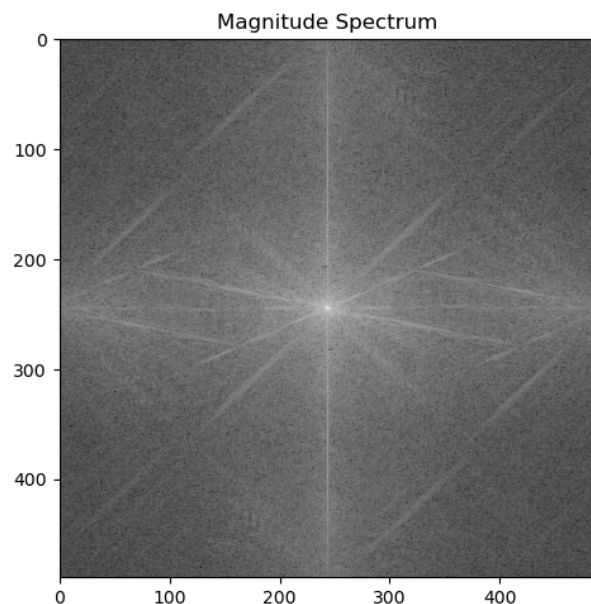
```

Gshift = f_transform_shifted * H
G = ifftshift(Gshift)
g = np.abs(ifft2(G))
return g

# Plotting function to compare original and filtered image
def plot_images(original, filtered, title):
    plt.figure(figsize=(12, 6))
    plt.subplot(1, 2, 1)
    plt.imshow(original, cmap='gray')
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(filtered, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.show()

```



a. Compare/comment on the output of Ideal, Butterworth and Gaussian High pass filters having the same radii (cutoff frequency) value.

```

# Set cutoff frequency
cutoff = 50

# Ideal High-Pass Filter
H_ideal = ideal_high_pass_filter(img1.shape, cutoff)
ideal_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_ideal)
plot_images(img1, ideal_filtered_img, f"Ideal High Pass Filter (D0={cutoff})")

# Butterworth High-Pass Filter (Order 2)
n = 2
H_butterworth = butterworth_high_pass_filter(img1.shape, cutoff, n)
butterworth_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_butterworth)
plot_images(img1, butterworth_filtered_img, f"Butterworth High Pass Filter (n={n}, D0={cutoff})")

# Gaussian High-Pass Filter
H_gaussian = gaussian_high_pass_filter(img1.shape, cutoff)
gaussian_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_gaussian)
plot_images(img1, gaussian_filtered_img, f"Gaussian High Pass Filter (D0={cutoff})")

# Ideal High-Pass Filter
H_ideal = ideal_high_pass_filter(img2.shape, cutoff)
ideal_filtered_img = apply_filter_and_inverse(f_transform_shifted, H_ideal)
plot_images(img2, ideal_filtered_img, f"Ideal High Pass Filter (D0={cutoff})")

```

```

# Butterworth High-Pass Filter (Order 2)
n = 2
H_butterworth = butterworth_high_pass_filter(img2.shape, cutoff,
n)
butterworth_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_butterworth)
plot_images(img2, butterworth_filtered_img, f"Butterworth High
Pass Filter (n={n}, D0={cutoff})")

# Gaussian High-Pass Filter
H_gaussian = gaussian_high_pass_filter(img2.shape, cutoff)
gaussian_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_gaussian)
plot_images(img2, gaussian_filtered_img, f"Gaussian High Pass
Filter (D0={cutoff})")

# Ideal High-Pass Filter
H_ideal = ideal_high_pass_filter(img3.shape, cutoff)
ideal_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_ideal)
plot_images(img3, ideal_filtered_img, f"Ideal High Pass Filter
(D0={cutoff})")

# Butterworth High-Pass Filter (Order 2)
n = 2
H_butterworth = butterworth_high_pass_filter(img3.shape, cutoff,
n)
butterworth_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_butterworth)

```

```

plot_images(img3, butterworth_filtered_img, f"Butterworth High
Pass Filter (n={n}, D0={cutoff})")

# Gaussian High-Pass Filter
H_gaussian = gaussian_high_pass_filter(img3.shape, cutoff)
gaussian_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_gaussian)
plot_images(img3, gaussian_filtered_img, f"Gaussian High Pass
Filter (D0={cutoff})")

```

Original Image



Ideal High Pass Filter (D0=50)



Original Image



Butterworth High Pass Filter (n=2, D0=50)



Original Image



Gaussian High Pass Filter ($D_0=50$)



Original Image



Ideal High Pass Filter ($D_0=50$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=50$)



Original Image



Gaussian High Pass Filter ($D_0=50$)



Original Image



Ideal High Pass Filter ($D_0=50$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=50$)



Original Image



Gaussian High Pass Filter (D0=50)



b. Consider a suitable gray scale image and demonstrate the ringing effect on the output of Ideal high pass frequency domain filter.

```
# Apply Ideal High-Pass Filter with ringing effect demonstration
cutoff = 30 # Sharp cut-off frequency to highlight ringing
H_ideal_ringing = ideal_high_pass_filter(img1.shape, cutoff)
ideal_filtered_img_ringing =
apply_filter_and_inverse(f_transform_shifted, H_ideal_ringing)
plot_images(img1, ideal_filtered_img_ringing, f"Ideal High Pass
Filter - Ringing Effect (D0={cutoff})")

H_ideal_ringing = ideal_high_pass_filter(img2.shape, cutoff)
ideal_filtered_img_ringing =
apply_filter_and_inverse(f_transform_shifted, H_ideal_ringing)
plot_images(img2, ideal_filtered_img_ringing, f"Ideal High Pass
Filter - Ringing Effect (D0={cutoff})")

H_ideal_ringing = ideal_high_pass_filter(img3.shape, cutoff)
ideal_filtered_img_ringing =
apply_filter_and_inverse(f_transform_shifted, H_ideal_ringing)
```

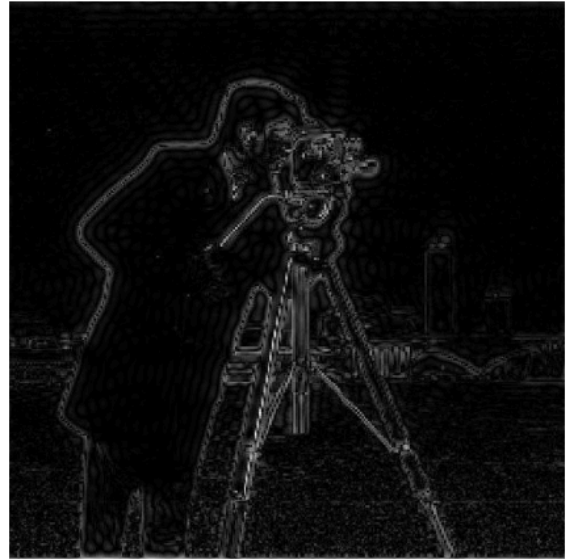


```
plot_images(img3, ideal_filtered_img_ringing, f"Ideal High Pass  
Filter - Ringing Effect (D0={cutoff})")
```

Original Image



Ideal High Pass Filter - Ringing Effect ($D_0=30$)



Original Image



Ideal High Pass Filter - Ringing Effect ($D_0=30$)



Original Image



Ideal High Pass Filter - Ringing Effect (D0=30)



c. Compare the output of Butterworth high pass filters (order $n=2$) for different cut-off frequencies (5, 15, 30, 90, 120).

```
cutoffs = [5, 15, 30, 90, 120]
n = 2    # Fixed order of the filter

# Compare Butterworth High-Pass Filters for different cutoff
frequencies
for c in cutoffs:
    H_butterworth = butterworth_high_pass_filter(img1.shape, c,
n)
                                butterworth_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_butterworth)
    plot_images(img1, butterworth_filtered_img, f"Butterworth
High Pass Filter (n={n}, D0={c})")

for c in cutoffs:
    H_butterworth = butterworth_high_pass_filter(img2.shape, c,
n)
```

```
                butterworth_filtered_img =  
apply_filter_and_inverse(f_transform_shifted, H_butterworth)  
                plot_images(img2, butterworth_filtered_img, f"Butterworth  
High Pass Filter (n={n}, D0={c})")  
  
for c in cutoffs:  
    H_butterworth = butterworth_high_pass_filter(img3.shape, c,  
n)  
  
                butterworth_filtered_img =  
apply_filter_and_inverse(f_transform_shifted, H_butterworth)  
                plot_images(img3, butterworth_filtered_img, f"Butterworth  
High Pass Filter (n={n}, D0={c})")
```

Original Image



Butterworth High Pass Filter ($n=2$, $D_0=5$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=15$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=30$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=90$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=120$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=5$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=15$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=30$)



Original Image



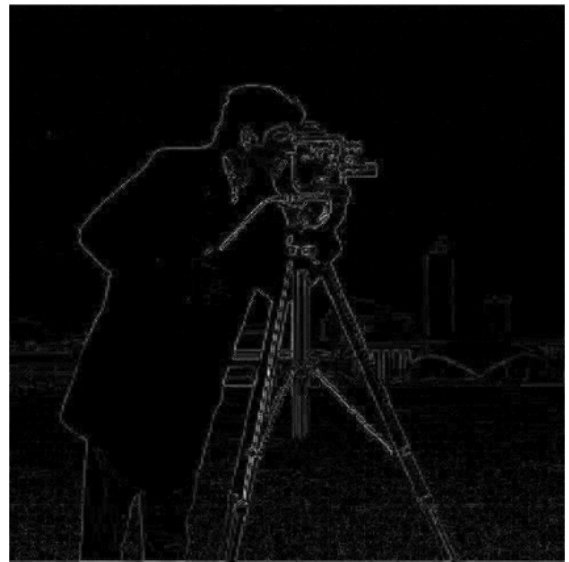
Butterworth High Pass Filter ($n=2$, $D_0=90$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=120$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=5$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=15$)



Original Image



Butterworth High Pass Filter ($n=2$, $D_0=30$)



Original Image



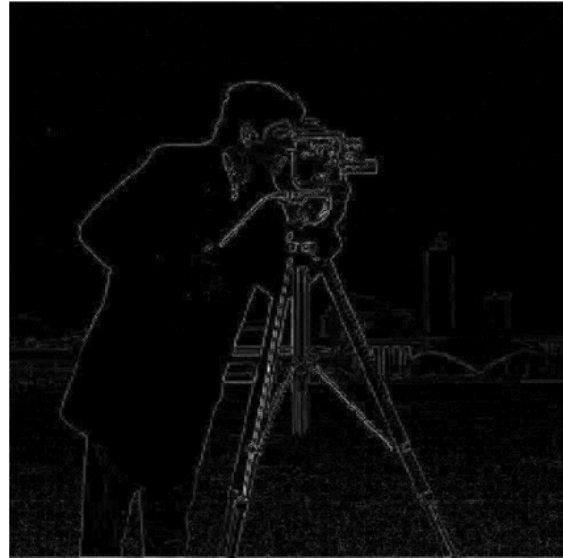
Butterworth High Pass Filter ($n=2$, $D_0=90$)



Original Image



Butterworth High Pass Filter (n=2, D0=120)



d. Compare the output of Gaussian high pass filters for different cut-off frequencies (5, 15, 30, 90, and 120).

```
cutoffs = [5, 15, 30, 90, 120]
n=2

# Compare Gaussian High-Pass Filters for different cutoff
frequencies
for c in cutoffs:
    H_gaussian = gaussian_high_pass_filter(img1.shape, c)
    gaussian_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_gaussian)
    plot_images(img1, gaussian_filtered_img, f"Gaussian High
Pass Filter (D0={c})")

for c in cutoffs:
    H_gaussian = gaussian_high_pass_filter(img2.shape, c)
    gaussian_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_gaussian)
    plot_images(img2, gaussian_filtered_img, f"Gaussian High
Pass Filter (D0={c})")
```

```

for c in cutoffs:
    H_gaussian = gaussian_high_pass_filter(img3.shape, c)
    gaussian_filtered_img =
apply_filter_and_inverse(f_transform_shifted, H_gaussian)
    plot_images(img3, gaussian_filtered_img, f"Gaussian High
Pass Filter (D0={c})")

```

Original Image



Gaussian High Pass Filter (D0=5)



Original Image



Gaussian High Pass Filter (D0=15)



Original Image



Gaussian High Pass Filter ($D_0=30$)



Original Image



Gaussian High Pass Filter ($D_0=90$)



Original Image



Gaussian High Pass Filter ($D_0=120$)



Original Image



Gaussian High Pass Filter ($D_0=5$)



Original Image



Gaussian High Pass Filter ($D_0=15$)



Original Image



Gaussian High Pass Filter ($D_0=30$)



Original Image



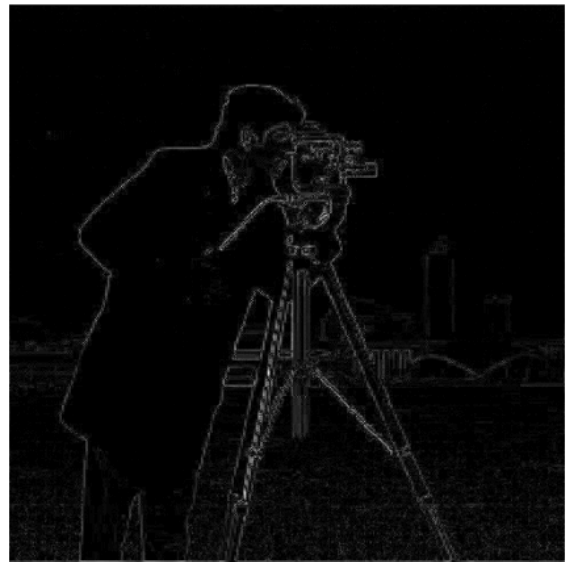
Gaussian High Pass Filter ($D_0=90$)



Original Image



Gaussian High Pass Filter ($D_0=120$)



Original Image



Gaussian High Pass Filter ($D_0=5$)



Original Image



Gaussian High Pass Filter ($D_0=15$)



Original Image



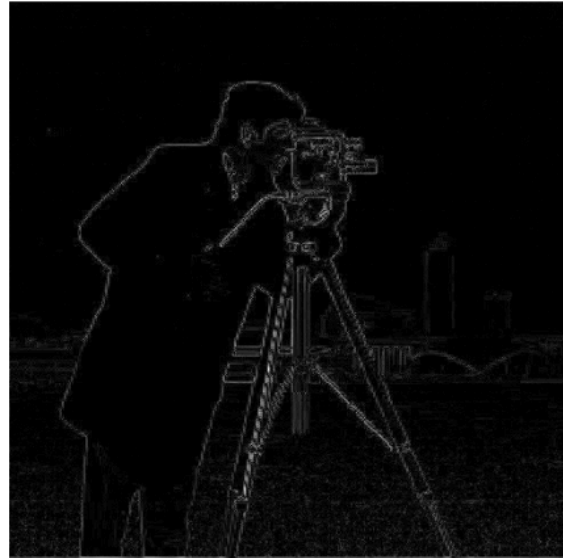
Gaussian High Pass Filter ($D_0=30$)



Original Image



Gaussian High Pass Filter ($D_0=90$)



Original Image



Gaussian High Pass Filter ($D_0=120$)

